Fat-Rat Technical Report

Application Description and Application Layer Protocol

Lebanese American University | CSC430 Computer Networks | Fall 2026

4 November 2026

This report specifies the planned behavior and communication protocol of Fat-Rat, a multiplayer quiz application using direct Python TCP and UDP sockets. It covers technical report sections 1 and 2. Implementation and experimental evidence will be documented in the later project sections.

1 Application Description

1.1 Purpose and participants

Fat-Rat is a real-time quiz application inspired by Kahoot. A host creates a quiz room, shares its PIN, and manages timed multiple-choice questions. Players join using the PIN and a username and compete for points awarded by the server. The host manages the game and does not compete. The host application is a client of the server and may run on a different device.

The shared host screen displays the question and its answer choices. Each answer is paired with a selection symbol. Player devices show the corresponding symbols, a countdown, answer-acceptance feedback and the player's cumulative score. They do not receive the active question text or answer text. This makes a screenshot of the player interface less useful for outside assistance, although it cannot prevent photographing the shared display. The protocol identifies eight distinct cheese-themed symbols by position; final icon artwork does not affect communication.

1.2 Preparing and joining a quiz

The host creates and edits quizzes through the application, saves them as JSON files and loads them for later use. Each quiz contains between 1 and 30 questions. Each question has 2 to 8 answer choices, exactly one correct choice, and a host-selected time limit of 1 to 300 seconds. The host loads the quiz locally and uploads its validated content to the server before starting.

The server issues a unique room PIN represented as a JSON string of six ASCII digits, including possible leading zeros. Players enter that PIN and a username of up to 32 Unicode code points after normalization and trimming; the server rejects invalid PINs and duplicate names within the room. Names are trimmed, NFC-normalized and compared without case distinctions. A late joiner starts at zero points and may answer the current open question using its remaining time. Joining during a pause or between questions does not start a personal timer.

One server supports independent simultaneous rooms, each with separate players, quiz content, timers and scores. The required demonstration consists of at least three concurrent rooms with at least five players each, including a client on a different device over a reachable local network [1]. Each room admits at most 50 ACTIVE players and starts with at least five connected ACTIVE players, excluding the host. Disconnected ACTIVE players retain a slot; permanent departure releases it while preserving history. After start, falling below five does not stop the quiz. Each host manages one unfinished room; multiple simultaneous rooms require separate hosts. Supported capacity will be established through testing in Section 4.

1.3 Question flow and scoring

The host starts the first question. The server opens its answer window and controls the deadline. A player may submit one valid answer per question attempt; once accepted, the answer is final. The client disables further submissions, while the server independently rejects invalid, duplicate or late requests. Acceptance feedback

reveals neither correctness nor points while answering remains open. The shared screen shows the live number of answered players.

A question closes when its time expires, every eligible player has answered, or the host closes it manually. Correctness, earned points and rankings are then revealed. The host explicitly advances to the next question. After the final question's results, advancing finishes the session and displays the complete final ranking.

For a correct accepted answer, the score is rounded to the nearest integer, with half-points rounded upward:

$$\text{Points} = \text{round half up}\left(500 + 500 \times \frac{\text{remaining time}}{\text{question time limit}}\right)$$

Both times use the same unit. A wrong answer or no accepted answer earns zero. For a 20-second question, correct answers accepted after 5, 10 and 15 seconds earn 875, 750 and 625 points respectively. The server measures time; a player cannot supply their own timing or score. Paused time does not reduce the speed bonus.

Players with equal cumulative scores share a single leaderboard row. Distinct scores receive consecutive ranks: for example, first place may list two names, followed by second and third place. The final leaderboard includes all participants and their scores, including retained records for players who left or were removed.

1.4 Host controls and live editing

A manual pause freezes the remaining time and stops answer acceptance while preserving existing submissions. Resuming continues with that saved duration. Editing the current question first pauses it and notifies players. Committing the edit clears all submissions for the discarded attempt, creates a new attempt and restores the full question time. The host explicitly resumes; changing the correct choice is safe because the discarded attempt earns no points. Cancelling the edit preserves the original question, answers and frozen time and leaves the quiz paused.

The host may edit, add, delete or reorder future questions during play. The current question's position and all completed questions are protected. These operations change the future question list without altering the current timer or accepted answers. Completed content may be edited again after the session finishes, but historical results remain unchanged. Playing a saved quiz again creates a fresh room with reset scores.

1.5 Connection failures and departures

A network disconnection preserves the player's identity, score and accepted answers. An unanswered disconnected player remains eligible for the current question, so disconnection does not shorten its timer. They may reconnect and answer before the deadline; otherwise that question earns zero. A locally stored random reconnect token restores the session without an account. When host disconnection is detected, an open question pauses; the host reconnects and explicitly resumes. Detection of a silent failure is not instantaneous.

A confirmed Leave room action permanently ends that identity's participation. Host removal does the same and attempts to send a clear removal notice. In both cases the server revokes the existing token, retains earned points and already-accepted answers, reserves the name, and excludes the departed player from pending-answer counts. A lost notice does not undo removal. Without accounts, entry under a different new name cannot be reliably prohibited. Rooms and reconnect state are retained in server memory until server shutdown; a host who never returns leaves the room in its current phase, paused if a question was open. Server crash recovery is outside this design.

2 Application Layer Protocol

2.1 Architecture and transport choices

The server is authoritative for membership, phase, timing, answer acceptance and scoring. Clients use one persistent TCP connection and one UDP socket. The implementation uses Python socket and threading facilities without HTTP, WebSocket or Socket.IO abstractions [1]. TCP carries commands and authoritative outcomes; UDP carries replaceable live display snapshots. TCP offers reliable ordered bytes, but application framing and

acceptance replies are still required [2, 4]. UDP can lose, duplicate or reorder datagrams, so the application handles those properties explicitly [3].

Category	Transport	Justification
Creation, joining, reconnecting and leaving	TCP	Identity, membership and permanent departure need explicit outcomes.
Quiz upload, edits and host controls	TCP	Ordered, complete commands prevent conflicting quiz state.
Question setup and phase changes	TCP	Clients must use the authoritative question attempt and phase.
Answers and acceptance replies	TCP	Lost or uncertain acceptance affects scoring and must be recoverable.
Results, rankings, removal and state recovery	TCP	Authoritative results and access decisions must be delivered consistently.
Liveness checks	TCP	UDP loss alone must not be treated as a disconnected host or player.
Endpoint registration	TCP and UDP	TCP supplies a challenge; a UDP proof identifies the actual receiving endpoint.
Countdown and answered counts	UDP	Each complete new snapshot replaces obsolete display information.

Example configurable ports are TCP 43000 and UDP 43001. Both sockets use IPv4. Every client, including one on the server machine, uses the same advertised server IPv4 address for TCP and UDP. The server binds UDP to that address so the source address is predictable. A loopback-only test uses 127.0.0.1 for both transports. Internet access and NAT traversal are not required. Traffic is plaintext on a trusted LAN; protection against interception and server-crash recovery are outside scope.

2.2 Message encoding and boundaries

All messages are UTF-8 JSON objects with v = 1, a type string and a data object. Unknown client fields, wrong types, duplicate keys, nonfinite numbers and invalid identifiers are rejected. JSON is encoded compactly without escaping ordinary Unicode text. IDs are UUIDv4 strings except the PIN, reconnect token and UDP nonce. Durations are integer milliseconds; revisions, sequence numbers and other numeric fields are nonnegative integers bounded by 2^63 - 1.

A TCP message consists of a four-byte unsigned big-endian length N followed by exactly N JSON bytes. The length excludes the prefix and must be between 1 and 1,048,576 bytes. The receiver repeatedly reads until it has the complete header and body. This handles partial reads and multiple messages in the stream without losing boundaries. Sending uses an ordered writer and sendall. A partial frame has a 10-second deadline from its first byte. EOF, including EOF in the middle of a frame, ends the connection [4].

A UDP datagram contains one complete JSON object with no length prefix and a maximum payload of 512 bytes. Oversized, truncated or malformed datagrams are dropped. The server sends about one LIVE_STATUS snapshot per second to each confirmed binding while the phase is OPEN or PAUSED. Snapshots contain no quiz text, scores, correctness or reconnect tokens. They provide meaningful live display updates; normal operation does not duplicate them as a per-second TCP stream.

Message kind	Additional envelope fields
Client TCP request	request_id; room_id after attachment. CREATE_ROOM and JOIN_ROOM are unattached; RESUME_SESSION puts room_id in data.

Message kind	Additional envelope fields
RESULT or ERROR	reply_to matching request_id; room_id when known. ERROR reply_to may be null when no usable request ID exists.
STATE	room_id and state_rev.
PING and PONG	No request_id, reply_to or room_id; data.ping_id identifies the exchange.
Other TCP session notices	room_id and the notice data listed in the catalogue.
UDP_HELLO	session_id and binding_id; data.nonce proves the challenge.
LIVE_STATUS	room_id, session_id, binding_id, question_id, attempt_id, control_rev and seq.

actor_id is a stable host/player identity; a player's player_id is that same value. session_id identifies one TCP attachment and changes on reconnect. attempt_id identifies one presentation of a question and changes when current-question editing restarts it. state_rev orders reliable snapshots. control_rev changes for quiz controls and phase changes, but not a player leaving or ordinary connectivity changes. Leaving updates state_rev; an automatic close caused by leaving also advances control_rev. An actual host removal advances both revisions; joining, voluntary leave and player connectivity advance state_rev only unless they cause an automatic close. In-room host mutations include expected_control_rev, preventing a delayed command from executing in a newer control state. UDP must match the latest TCP-confirmed control revision; state_rev alone is not used to validate a tick.

2.3 Shared data structures

A Question contains question_id, text, choices, correct_choice_id and time_limit_ms. Question, choice and title text must be nonempty. choices is an ordered array of {choice_id, text}; question IDs are unique and choice IDs are unique within a question. The host generates these IDs, while the server validates them. choices contains 2 to 8 entries, correct_choice_id must identify one of them, and time_limit_ms must be an integer from 1,000 to 300,000; all text fields must be nonempty after trimming. Player question setup contains {question_id, attempt_id, choices:[{choice_id, symbol_index}], time_limit_ms}; symbol_index is the zero-based choice position from 0 to 7. The host uses the same ordering with the full text.

A RankRow contains {rank, score, players:[{player_id, username, participation}]}. participation is ACTIVE, LEFT or REMOVED; disconnection does not change ACTIVE status. A QuestionResult contains {question_id, attempt_id, correct_choice_id, reason, own_result}. reason is ALL_ANSWERED, TIMEOUT or HOST_CLOSED. A player's own_result contains {choice_id or null, correct, points, total_score}; the host's own_result is null. Results refer to frozen closed attempts; STATE.leaderboard is the single ranking. On FINISHED, the full final ranking and its participation labels are frozen. Subsequent departure can revoke a token without rewriting that ranking.

The reusable quiz list is separate from each immutable current/historical attempt. This prevents post-session edits from changing completed choices or scoring. Technical safety limits are 1,000 characters per question text, 300 per choice, 100 per title, and 512 KiB for compact quiz JSON. Every quiz mutation and join is checked against the full 1 MiB outgoing-state budget, including roster, frozen question and result overhead. Character limits count Unicode code points. These are application limits, not limits imposed by TCP or UDP.

2.4 Message catalogue

H denotes the host client, P a player client, C either client and S the server. Every request receives RESULT or ERROR. The following rows define data fields in addition to the common envelope. Generic successful commands return an empty data object; identity and answer responses are defined explicitly. For a requester, RESULT precedes the resulting STATE. Only STATE changes a client's authoritative phase.

Room membership and participation

Message	Direction and transport	Data fields and behavior
CREATE_ROOM	H to S TCP	{username}. Create an empty LOBBY and host identity; RESULT returns identity fields below. The cross-room host identity check and release operation remain pending clarification; this draft catalogue is not complete for that rule.
JOIN_ROOM	P to S TCP	{pin, username}. Join LOBBY, OPEN, PAUSED or RESULTS; reject new joins to FINISHED. Reject at 50 ACTIVE players with ROOM_FULL; disconnected ACTIVE players occupy a slot, LEFT/REMOVED history does not. RESULT returns identity; then role-filtered STATE.
RESUME_SESSION	C to S TCP	{room_id, reconnect_token}. Authenticate retained identity; room_id is in data because connection is not yet attached. RESULT returns identity; then STATE.
REMOVE_PLAYER	H to S TCP	{expected_control_rev, player_id}. Mark REMOVED, revoke token, preserve score/accepted answers and exclude from eligibility. Send best-effort REMOVED notice. Cannot target host. If already LEFT or REMOVED, successful no-op: RESULT only; preserve original participation/revocation reason and revisions.
LEAVE_ROOM	P to S TCP	{}. Sent only after client confirms permanent departure. Atomically mark LEFT, revoke token, preserve score/accepted answers and exclude from eligibility. RESULT {left:true} then close the departing TCP connection. No notification delivery guarantee is required to enforce departure.
GET_STATE	C to S TCP	{}. RESULT {} then a complete role-filtered STATE. Used on reconnect uncertainty, stale revisions or prolonged lost UDP.

Quiz content and host controls

Message	Direction and transport	Data fields and behavior
PUT_QUIZ	H to S TCP	{expected_control_rev, title, questions:[Question]}. Replace quiz atomically in LOBBY only; never starts it. Loading/saving local JSON is a client action; the host sends this validated quiz to S.
START_QUIZ	H to S TCP	{expected_control_rev}. LOBBY with a valid loaded quiz and at least five connected ACTIVE players, excluding the host, to OPEN first question; missing quiz returns INVALID_STATE, insufficient players returns NOT_ENOUGH_PLAYERS.
SUBMIT_ANSWER	P to S TCP	{question_id, attempt_id, choice_id}. No claimed elapsed time, score, username or player_id; identity comes from TCP session. RESULT {accepted:true, question_id, attempt_id, choice_id}; no correctness/points.
PAUSE_QUIZ	H to S TCP	{expected_control_rev}. OPEN to PAUSED, reason MANUAL; retain submissions and remaining time.
RESUME_QUIZ	H to S TCP	{expected_control_rev}. PAUSED to OPEN if no edit is active; use stored remaining time. Current edit commit already reset that stored time to the full limit.
CLOSE_QUESTION	H to S TCP	{expected_control_rev}. OPEN or PAUSED not editing to RESULTS; score once; do not close while editing is active.
NEXT_QUESTION	H to S TCP	{expected_control_rev}. RESULTS to next question OPEN, or FINISHED after final results. No skipping/reopening completed questions.

Message	Direction and transport	Data fields and behavior
EDIT_BEGIN	H to S TCP	{expected_control_rev, question_id}. Current uncompleted question only in OPEN/PAUSED; freeze if OPEN, mark editing active, preserve original content, submissions and frozen time. Not a content mutation yet.
EDIT_COMMIT	H to S TCP	{expected_control_rev, question:Question}. During active current edit, same question ID; validate then replace atomically, new attempt ID, clear all submissions, full stored time, PAUSED with editing false. Host explicitly RESUME_QUIZ. Revalidate full quiz and outgoing message byte budgets before commit.
EDIT_CANCEL	H to S TCP	{expected_control_rev}. End current editing without changing content, submissions or attempt; remain PAUSED with original frozen time; explicit resume.
EDIT_QUESTION	H to S TCP	{expected_control_rev, question:Question}. Replace a future question in an active session; or any question in LOBBY/FINISHED. Same question ID and position. Completed question edit during active session to QUESTION_LOCKED. FINISHED edits do not change saved result snapshots.
ADD_QUESTION	H to S TCP	{expected_control_rev, question:Question, before_question_id:id or null}. Insert a question whose ID is absent from the current quiz before an editable future target, or append if null. Active session: never insert at/before current or completed question; LOBBY/FINISHED: entire reusable list is editable. Limit remains 30.
DELETE_QUESTION	H to S TCP	{expected_control_rev, question_id}. Delete only a future question during an active session; LOBBY/FINISHED may delete any reusable question. Keep at least one reusable question. Never alter the current attempt or historical results.
REORDER_ QUESTIONS	H to S TCP	{expected_control_rev, question_ids:[id]}. Complete ordered permutation of the editable suffix, or full reusable list in LOBBY/FINISHED. Reject missing/extra/duplicate IDs or inclusion of locked prefix.

Responses and live communication

Message	Direction and transport	Data fields and behavior
REQUEST_UDP_BIND	C to S TCP	{}. RESULT {binding_id, udp_nonce, server_udp_port}; replaces old binding.
UDP_HELLO	C to S UDP	{nonce} plus session_id and binding_id envelope. S validates challenge and same IP as TCP peer, records observed source port, replies UDP_BOUND over TCP.
UDP_BOUND	S to C TCP	{binding_id}. Confirms endpoint; only then send/apply LIVE_STATUS for this binding.
LIVE_STATUS	S to C UDP	{remaining_ms, answered_count, eligible_count}. Counts for the current attempt, including unanswered disconnected participants. No choices/correctness/scores.
RESULT	S to C TCP	Data specific to the request. Generic successful mutation: {}. Create/join/resume: {room_id, pin, session_id, actor_id, role, reconnect_token} where role is HOST/PLAYER.

Message	Direction and transport	Data fields and behavior
ERROR	S to C TCP	{code, message, retryable}; reply_to null only if no usable request ID. Error-specific fields in data: current_control_rev for STALE_STATE; required reason for SESSION_REVOKED; question_id/attempt_id/choice_id for ALREADY_ANSWERED; retry_after_ms for RATE_LIMITED or SERVER_BUSY. Never include tokens/correct answers.
STATE	S to C TCP	Complete, role-filtered snapshot defined below. Sent after identity, GET_STATE and reliable room transitions, to recipients specified below.
PING	S to C TCP	{ping_id} random UUID, one outstanding per connection. Send a new ping five seconds after the previous valid PONG (or attachment), only if none is outstanding. PONG deadline is 15 seconds after enqueue; slow writer queues are independently bounded. Respond immediately with matching PONG.
PONG	C to S TCP	{ping_id} matches outstanding ping. Session event, not a request; no request_id/result.
REMOVED	S to P TCP	{reason:"HOST_REMOVED", message:string}. Best-effort final notice followed by disconnect; removal is committed even if delivery fails.
SESSION_REPLACED	S to C TCP	{reason:"RECONNECTED_ELSEWHERE"}. Best-effort notice to old connection after valid session resume replaces it.

2.5 State snapshots and visibility

STATE data contains control_rev, phase, pause_reason, editing, host_connected, title, question_index, question_count, current_question, remaining_ms, self, leaderboard, last_result and live. question_index is zero-based or null before start. phase is LOBBY, OPEN, PAUSED, RESULTS or FINISHED. pause_reason is null unless PAUSED, when it is MANUAL, EDITING or HOST_DISCONNECTED. editing is true exactly for an EDITING pause. EDIT_COMMIT and EDIT_CANCEL set pause_reason to MANUAL. host_connected is independent, so editing can remain active while the host is disconnected.

self contains {actor_id, role, username, total_score, submitted_choice_id}; score and choice are null for the host. Host/player names must contain 1 to 32 Unicode code points after NFC normalization and trimming, with INVALID_USERNAME on failure; only player names reserve room membership. A player receives only the symbol-based current_question structure. The host additionally receives quiz (null before loading, otherwise {title, questions}) and participants:[{player_id, username, connected, participation, total_score}]. Host current_question also contains the question and choice text. The shared presentation hides correct-answer/editor metadata. Correct answer IDs never appear in player state before that attempt closes.

leaderboard contains cumulative scores from completed questions, with zero-score rows initially. last_result is null or the most recently closed QuestionResult, privately projected for its recipient. live is always present: null in ordinary STATE or, in active recovery snapshots, {answered_count, eligible_count}. Each player receives their own serialized snapshot; one player's self or own_result must never be broadcast to others.

A control_rev or host_connected change sends fresh STATE to the attached host and every attached ACTIVE player. Joining, removal and voluntary leave also update all remaining ACTIVE clients. A player-only disconnect/reconnect updates the host and reconnecting player; it need not notify other players of connectivity alone. GET_STATE replies only to its requester. STATE is not broadcast for every accepted answer: that acceptance is private and UDP carries live counts. Clients accept increasing state_rev values and equal-revision recovery refreshes; older snapshots are ignored.

2.6 Timing and valid transitions

The server measures now_ms = time.monotonic_ns() // 1,000,000. This clock is not affected by wall-clock adjustments [5]. An OPEN attempt has deadline_ms; remaining_ms = max(0, deadline_ms - now_ms). An answer is late when now_ms is at or after that deadline. The authoritative acceptance time is sampled after acquiring the room lock, rather than trusting a client timestamp or first-byte arrival. This policy has ordinary network and scheduling latency effects; it does not claim perfect latency fairness.

Every serialized handler first closes an overdue OPEN attempt. The timer thread performs the same deadline check under the same lock, so late timer scheduling cannot admit a late answer. Pause stores remaining_ms, and resume sets deadline_ms to now_ms plus that saved duration. Accepted submissions record their remaining time; points are awarded exactly once when the attempt closes. With L = time_limit_ms and r = accepted remaining milliseconds, the exact half-up score for a correct answer is 500 + (1000r + L) // (2L). A new answer at r = 0 is rejected.

Phase and action	Result
LOBBY plus START_QUIZ	OPEN first question with a new attempt and full timer; a valid quiz and permitted player count are required.
OPEN plus PAUSE_QUIZ	PAUSED with submissions and frozen time preserved.
OPEN or PAUSED plus EDIT_BEGIN	PAUSED with editing active; preserve original question, submissions and frozen time.
Editing plus EDIT_COMMIT	PAUSED with editing inactive, a new attempt, no submissions and full time. No score for the discarded attempt.
Editing plus EDIT_CANCEL	PAUSED with original question, attempt, submissions and time unchanged.
PAUSED plus RESUME_QUIZ	OPEN with saved time, or directly RESULTS if all remaining eligible players have already answered.
OPEN plus deadline or all answered	RESULTS with immutable correctness, points and rankings.
OPEN or nonediting PAUSED plus CLOSE_QUESTION	RESULTS; score existing accepted answers, unanswered zero.
RESULTS plus NEXT_QUESTION	OPEN the next current future-list entry, or FINISHED if none remains.
Host failure while OPEN	PAUSED when detected, unless the deadline already expired; explicit host resume required.

Eligible players are all ACTIVE participants, including unanswered disconnected players and late joiners. A disconnection does not reduce this set. Explicit leave/removal does. Early close requires a nonempty eligible set in which everyone has answered; an empty room keeps its timer until expiry or manual close. Recheck all-answered after answers and eligibility changes, and on resume after departures during a pause. Live answered_count includes only accepted answers from eligible players, so it never exceeds eligible_count.

2.7 Current and future question editing

While OPEN, PAUSED or RESULTS, let k identify the current or just-closed question. Indices 0 through k are protected; only the suffix after k can be added to, deleted from or reordered. Current-question content changes use the EDIT_BEGIN and EDIT_COMMIT transaction. ADD_QUESTION inserts before a future ID or appends when its target is null. DELETE_QUESTION removes a future ID. REORDER_QUESTIONS supplies every current future ID exactly once, with no omissions or extra IDs. Question IDs must be unique within the candidate quiz; a deleted future question may be re-added with its saved ID.

The server validates the complete candidate quiz and outgoing size budget before committing a change. Each actual content/list change advances the revisions and reliably refreshes every ACTIVE client's private STATE. It never changes the current attempt, timer or submissions unless it is EDIT_COMMIT. If a list change races NEXT_QUESTION, room-lock order and expected_control_rev allow one operation to succeed; the other synchronizes rather than accidentally modifying a newly current question. Deleting all future questions makes the current one final; appending before advancement makes the added question playable.

In LOBBY and FINISHED, the reusable list is fully editable, with 1 to 30 questions. After FINISHED, question_index, question_count, current_question and recorded results remain the frozen history of that session even if reusable quiz content changes. A new room is required to replay it. An identity reorder or reusable question/quiz replacement identical after canonical compact JSON serialization with recursively sorted object keys and preserved array order is a no-op: RESULT only, unchanged revisions and no cache entry. EDIT_COMMIT still restarts even when its text is identical. EDIT_CANCEL always stays paused; no editor action implicitly resumes play.

2.8 UDP registration and recovery

After attaching over TCP, the client sends REQUEST_UDP_BIND. The server returns a new binding_id, a random 128-bit UDP nonce encoded as 32 lowercase hexadecimal characters, and server_udp_port. It disables the old binding. The client sends UDP_HELLO from its receiving UDP socket to that port. The server checks the session, challenge, binding and TCP-peer source IP, records the observed source port, and confirms with UDP_BOUND over TCP. The client discards snapshots until that confirmation. Reconnection uses a new session_id and repeats this binding exchange; the reconnect token never travels over UDP.

Challenges last 10 seconds. HELLO retries are limited to once per second, at most five sends before renewing over TCP with backoff capped at 30 seconds. A matching HELLO from the same endpoint can be confirmed again harmlessly; changing the source port needs a fresh binding. Unsolicited or invalid datagrams receive no response.

The server reads an entire LIVE_STATUS snapshot atomically under the room lock, then sends outside it. Fields include the room, attachment, binding, question, attempt, control revision, sequence, remaining time and answered/eligible counts. In PAUSED, time is the stored frozen duration. The client checks server address/port, current binding, exact room/question/attempt/control revision, and a sequence greater than the last accepted one. Sequence starts at 1 for each binding; the client resets its last accepted value to 0 when that binding is confirmed. Packets for older or not-yet-confirmed future control states are discarded. A delayed pre-pause update cannot revive a timer after resuming because control_rev changed.

An isolated lost packet is tolerated: the next complete snapshot supersedes it, and the client estimates the countdown locally between updates. Duplicate and reordered snapshots fail the sequence test. UDP never changes phase or determines acceptance. After three seconds without a matching snapshot in OPEN/PAUSED, mark progress stale and request GET_STATE no more than once per five seconds; persistent loss triggers rebinding with backoff. No stale polling occurs in LOBBY, RESULTS or FINISHED. Loss of UDP alone never disconnects a player or pauses the host. The display remains approximate under delay; the server deadline is authoritative.

2.9 Errors retries and session recovery

Malformed lengths, incomplete frames, invalid UTF-8/JSON, duplicate keys, nonfinite numbers, non-object payloads and unsupported versions are fatal to that connection. Send a bounded ERROR when safe, then close; never guess a new frame boundary. Well-formed requests with schema or state errors get a correlated ERROR and leave the connection usable. Oversized/malformed UDP is silently dropped. An invalid client must not stop the server or unrelated rooms.

Error codes	Handling
INVALID_PIN / INVALID_USERNAME / NAME_TAKEN	Join invalid room, invalid host/player name, or reserved normalized player name. Stay unauthenticated.
INVALID_MESSAGE / UNSUPPORTED_VERSION	Envelope/schema/encoding/version violation; fatal behavior follows framing rules.
INVALID_QUIZ / INVALID_CHOICE	Invalid question set or selected choice; no mutation/answer acceptance.
NOT_AUTHENTICATED / NOT_HOST / NOT_PLAYER	Missing session, player attempting host action, or host submitting an answer/LEAVE_ROOM; no mutation.
ALREADY_ATTACHED / UNKNOWN_PLAYER	New create/join/resume on an attached connection; or removal targeting unknown/host identity. No mutation. REMOVE_PLAYER targeting an already LEFT or REMOVED player succeeds as a no-op without caching, state/status change, new notice or revision increment. The first committed departure reason is permanent.
ROOM_MISMATCH / ROOM_FINISHED	Attached session targets another room, or new join to FINISHED.
INVALID_TOKEN / SESSION_REVOKED	Unknown room or unknown token returns INVALID_TOKEN with the same generic message (including after server restart); no identity restored; a retained revoked token returns SESSION_REVOKED with required data.reason HOST_REMOVED or LEFT_ROOM, and a human-readable message. Other ERROR types omit reason unless specified. This is a denial response, not a replacement token.
STALE_STATE / STALE_ATTEMPT	Revision mismatch or wrong question/attempt; server sends fresh private STATE after either error; inspect it before deciding next action.
ROOM_FULL / NOT_ENOUGH_PLAYERS	Join would exceed 50 ACTIVE players, or START_QUIZ has fewer than five connected ACTIVE players. Host is excluded. Existing-player resume does not consume another slot. No mutation.
INVALID_STATE / QUESTION_LOCKED	Action not allowed in current phase, no quiz, or mutation of current/completed prefix through future-only operation. Use EDIT_BEGIN to edit current content.
QUESTION_LIMIT / INVALID_ORDER	Add would exceed 30 questions or delete would leave an empty reusable quiz; or reorder is not an exact valid permutation. No mutation.
ALREADY_ANSWERED	Different request tries another submission after acceptance.
QUESTION_PAUSED / ANSWER_LATE	New answer while paused, or server deadline reached; reject.
REQUEST_ID_REUSED / RATE_LIMITED / SERVER_BUSY	Conflicting recent request ID, excess requests, or configured resource/capacity reached; no mutation.

ERROR data contains code, message and retryable. retryable is true only for RATE_LIMITED or temporary SERVER_BUSY, optionally with retry_after_ms; a permanent byte-budget refusal is not retryable. STALE_STATE carries current_control_rev; a fresh private STATE follows either STALE_STATE or STALE_ATTEMPT. ERROR reason is required for SESSION_REVOKED; question_id, attempt_id and choice_id identify ALREADY_ANSWERED; retry_after_ms may accompany RATE_LIMITED or SERVER_BUSY. ALREADY_ANSWERED includes that player's accepted question, attempt and choice. A matching current closed attempt returns ANSWER_LATE; an older attempt after advancing returns STALE_ATTEMPT. Paused submissions return QUESTION_PAUSED. Invalid choice, unauthorized host actions and duplicate acceptance are rejected without changing scores.

Each logical request has a fresh request_id. After an uncertain connection failure, reconcile fresh STATE before issuing another action. The server retains each accepted answer's request fingerprint and outcome. An exact accepted-answer retry returns its historical acceptance and fresh STATE without re-awarding points or reviving a discarded attempt. A different submission follows normal duplicate rules. Answer and host-result fingerprints are keyed by (actor_id, request_id) and survive session replacement. For successful host mutations, retain the last 32 fingerprints and outcomes; a cached retry returns its outcome plus current STATE. An evicted original command cannot execute again because its expected_control_rev is stale. Reused cached IDs with different content are rejected. Errors and state/binding/identity recovery operations are not cached.

One room lock serializes commands, expiry, replay checks, answer insertion, scoring, departures and quiz-list changes. Identity and connection generation are rechecked inside that lock before executing or replaying anything. Commit immutable events in order, then write outside the lock through one bounded writer queue per connection. A slow client is disconnected instead of blocking a room. A 10-request-per-second attachment limit excludes PONG; recovery does not depend on an unbounded result cache. These rules support the required threaded implementation; detailed implementation belongs in report section 3.

The server sends PING five seconds after the previous valid PONG, allowing one outstanding ping. Missing PONG for 15 seconds, EOF, reset or write failure marks disconnection. A client treats 20 seconds without a complete TCP frame as stalled and attempts reconnect with 1, 2, 4, 8-second backoff capped at 15 seconds. A token consists of 32 cryptographically random bytes encoded as unpadded URL-safe Base64 (43 characters) and is scoped to one actor and room. The app stores it locally, separately from quiz content. Rooms and tokens remain in memory until server shutdown; completed rooms reject new joins but retained ACTIVE identities may resume to see final results.

A valid resume replaces the old TCP attachment and UDP binding, preserving the actor's scores and submissions. Old queued requests and cleanup callbacks fail generation checks. Replacing an apparently still-connected host first applies the same pause rule as disconnection, then installs the new attachment; reconnecting never automatically resumes. Silent failure may consume time before detection, and an expired attempt is not retroactively paused. Losing the initial create/join response before receiving a token cannot be securely recovered by username alone. An orphaned player remains ACTIVE and eligible until the host removes them; their reserved name is not released, so a tokenless rejoin needs another name. An orphaned host room remains until shutdown.

REMOVE_PLAYER commits REMOVED status and token revocation before attempting a final REMOVED notice. LEAVE_ROOM commits LEFT status and revocation before its final {left:true} result. Both preserve accepted answers for normal closing scores and remove eligibility. Notifications use the ordered writer with a bounded flush and then close; they never interrupt a partially written frame. Undelivered notification does not restore access. A retained revoked token receives SESSION_REVOKED with HOST_REMOVED or LEFT_ROOM reason on a fresh reconnect. The app then discards it. INVALID_TOKEN, including an unknown room after server restart, also stops automatic resume. If resume succeeds after an uncertain leave, leave did not commit and the client may submit a fresh LEAVE_ROOM using the original confirmation. Confirmed departure is displayed only after {left:true} or a revoked-token denial. Removing an already LEFT/REMOVED identity is a no-op that preserves its original departure reason. After FINISHED, token revocation cannot change the frozen final leaderboard. No replacement denial token is issued. A plain connection drop preserves ACTIVE status and reconnect rights.

2.10 Typical exchanges and sequence diagrams

Figure 1 shows room setup, player registration, UDP binding and one complete question. Solid arrows are TCP and dashed arrows are UDP. The host also binds its own UDP endpoint. Additional players use the same join sequence. Acceptance remains private until the server publishes results.

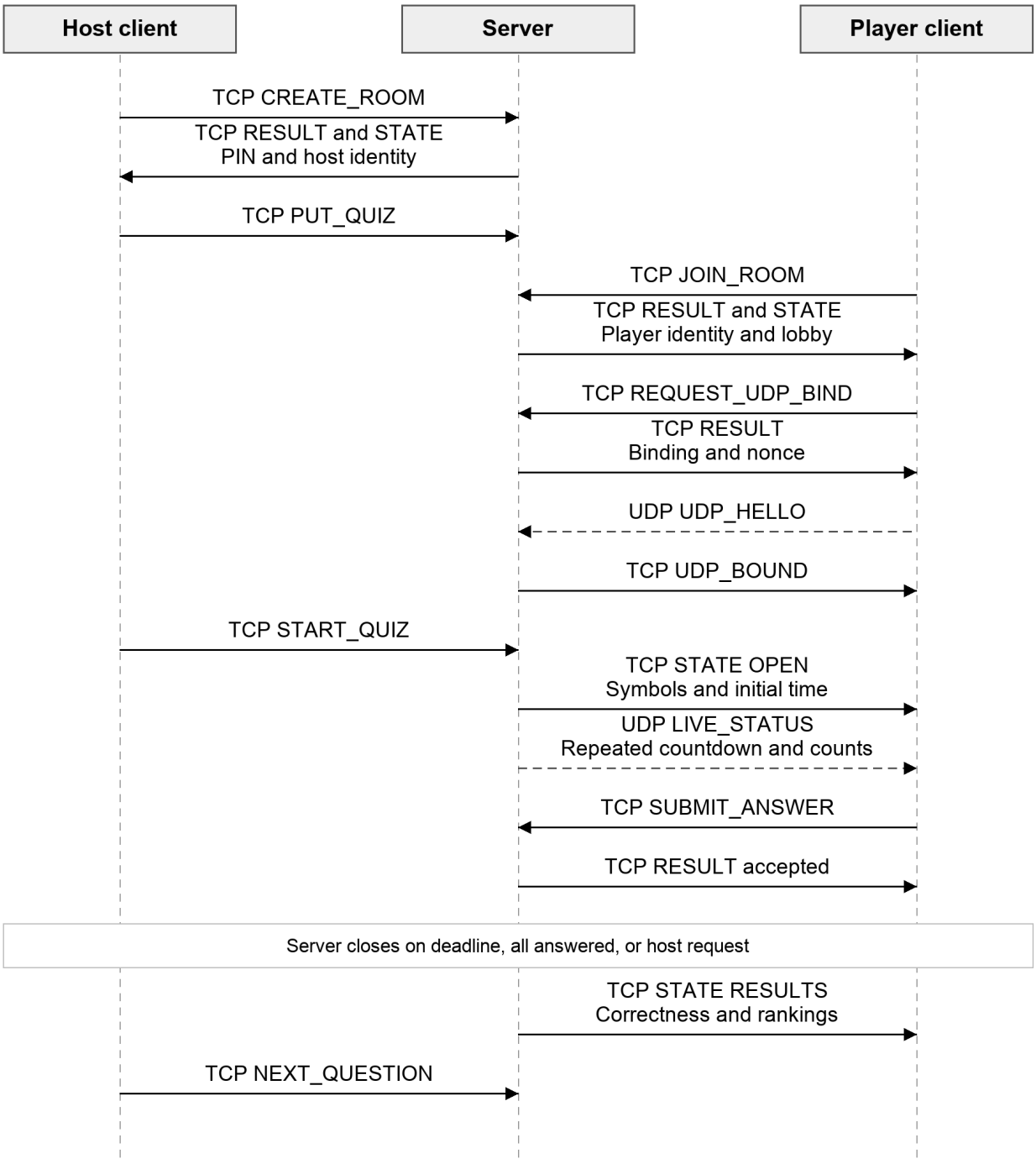


Figure 1 Typical room setup and question exchange. Host state changes are also delivered over TCP; live host progress uses its own UDP binding.

Figure 2 shows the current-question editing transaction and a player reconnecting after an uncertain answer response. A new attempt invalidates old submissions and UDP snapshots. Reconnection restores the server's accepted answer and establishes a fresh UDP binding.

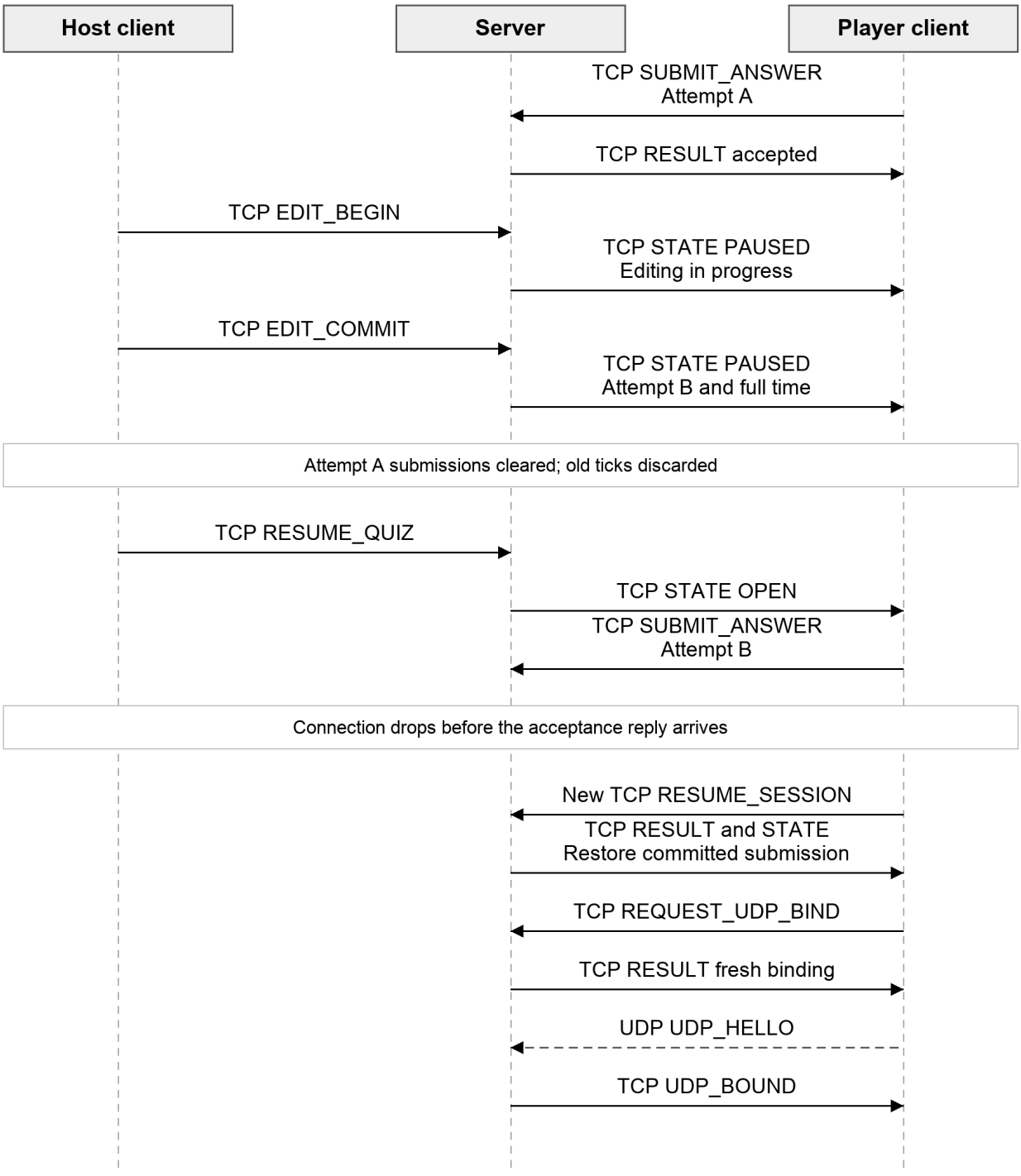


Figure 2 Current-question restart and reconnect recovery. The host explicitly resumes; the player reconciles STATE before another action.

2.11 Example application messages

The following answer request illustrates the JSON body. The four-byte TCP prefix is transmitted before it and is not printed here. IDs are valid example UUIDs, not live session credentials.

```
{
  "v": 1,
  "type": "SUBMIT_ANSWER",
  "request_id": "b3a4622d-a17c-43e6-8098-f00d59f3089b",
  "room_id": "3f9441d2-7ab1-47d2-b686-4c39745c46cb",
  "data": {
    "question_id": "3219d327-05b6-438c-a5dc-61387e3fb561",
    "attempt_id": "cbdd3abd-22b5-47be-a9b1-ce2c60a8ed15",
    "choice_id": "2de5e0c9-55b2-49b4-9429-4a61a3b9a2fb"
  }
}
```

The matching RESULT uses reply_to equal to request_id and data {accepted:true, question_id, attempt_id, choice_id}. It contains no correctness or points. A UDP LIVE_STATUS body instead uses remaining_ms, answered_count and eligible_count, accompanied by the binding, attempt, control revision and sequence envelope defined in section 2.2. A zero displayed countdown does not itself close the attempt; clients wait for authoritative STATE or an answer error.

References

[1] Lebanese American University. CSC430 Computer Networks, Fall 2026. Course Project: Live Quiz Arena Socket Programming. Project Requirements.pdf, supplied course specification.

[2] Eddy, W., editor. RFC 9293: Transmission Control Protocol. IETF, August 2022, section 2.2. https://www.rfc-editor.org/rfc/rfc9293.html

[3] Eggert, L., Fairhurst, G., and Shepherd, G. RFC 8085: UDP Usage Guidelines. IETF, March 2017, sections 3.2 and 3.3. https://www.rfc-editor.org/rfc/rfc8085.html

[4] Python Software Foundation. Socket Programming HOWTO. https://docs.python.org/3/howto/sockets.html

[5] Python Software Foundation. time.monotonic and time.monotonic_ns. https://docs.python.org/3/library/time.html#time.monotonic

AI assistance: OpenAI Codex and Anthropic Claude were used for protocol drafting, critical review and document preparation. The behavior described is a design specification; no implementation, load-test or Wireshark result is claimed in these sections.